

Architecture du projet

Paris Basketball — Analyse et prédiction des revenus par match
Documentation technique de bout en bout

Ce document décrit l'architecture complète mise en place pour ce projet : la circulation des données depuis les fichiers sources jusqu'aux tableaux de bord, l'infrastructure de traitement (cluster Spark conteneurisé), l'orchestration et la planification automatique, la journalisation, la modélisation en schéma en étoile, le modèle de prédiction, les deux applications de restitution (Streamlit et HTML autonome), et l'intégration d'un copilote IA via l'API Mistral. Chaque section explique le rôle du composant, comment il s'articule avec les autres, et pourquoi ce choix a été fait.

La figure de la section 1 présente une vue d'ensemble de l'interopérabilité entre tous les outils utilisés — c'est le point d'entrée recommandé pour parcourir ce document.

Document généré le 17/08/2026.

1. Vue d'ensemble : interopérabilité des outils

Le schéma ci-dessous montre comment circulent les données à travers l'ensemble des outils du projet, du fichier source brut jusqu'aux tableaux de bord consultés par l'utilisateur final. Chaque flèche représente un flux de données réel (fichier écrit, table lue, appel API) — aucun composant ne fonctionne isolément.

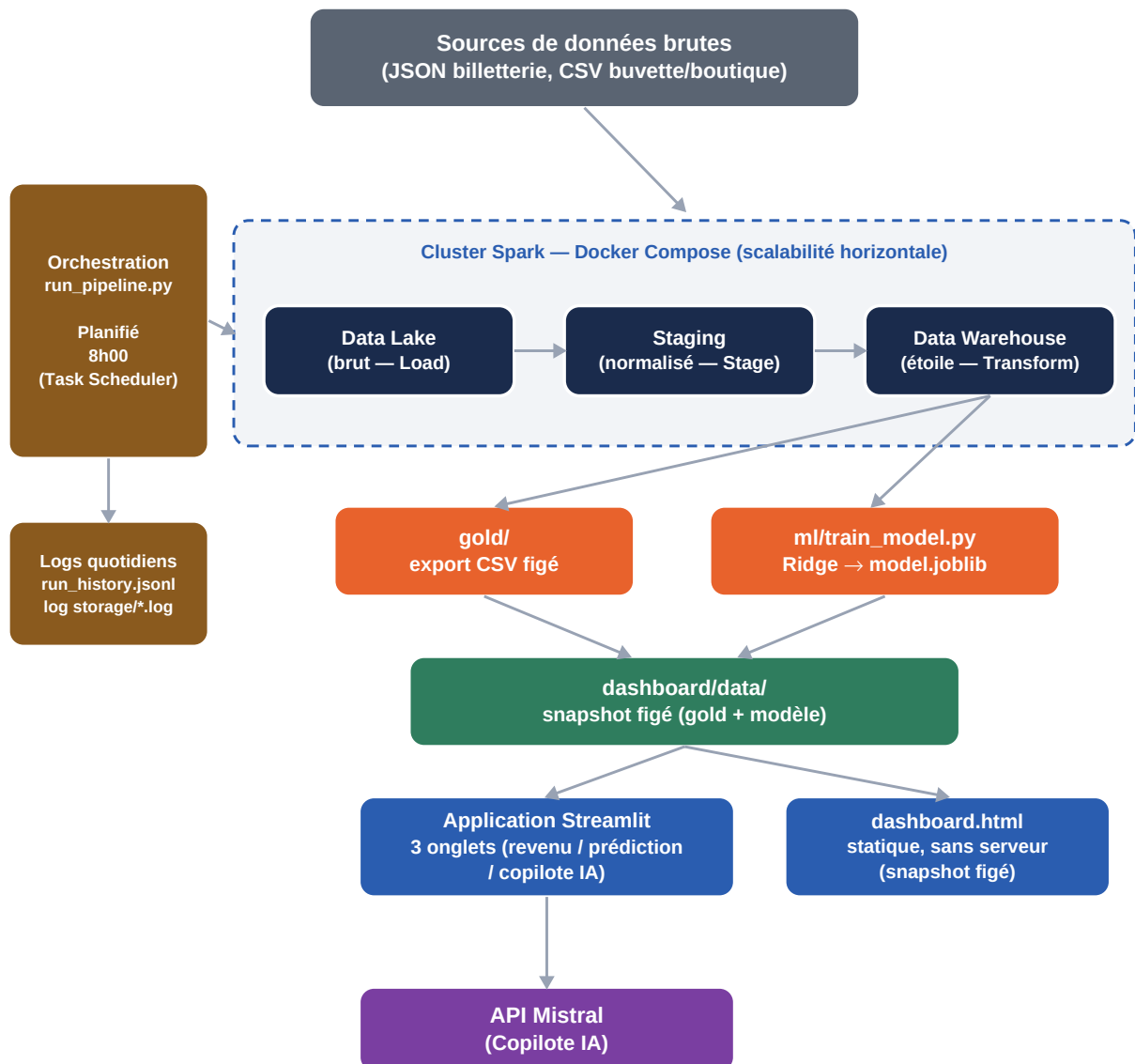


Figure 1 — Les sources brutes sont chargées et transformées sur le cluster Spark (conteneurs Docker Compose), piloté par l'orchestrateur planifié chaque jour à 8h00, qui journalise chaque exécution. Le Data Warehouse en sortie alimente à la fois un export CSV figé (gold/) et le réentraînement du modèle de prédiction ; les deux convergent dans un instantané (dashboard/data/) consommé indépendamment par le dashboard Streamlit (connecté à l'API Mistral pour le copilote IA) et par le dashboard HTML autonome.

2. Architecture data : Data Lake et Data Warehouse

Les données traversent quatre couches successives, chacune ayant un rôle précis. Cette séparation garde les données brutes intactes en cas d'erreur en aval, permet de corriger et normaliser progressivement, et ne construit le modèle d'analyse qu'une fois les données fiables.

Couche	Dossier	Rôle
Extraction	(en mémoire)	Récupère les données brutes depuis la source déclarée dans config/sources_config.py (archive locale ou API), sans rien écrire sur disque.
Load	Data Lake/	Dépose les données brutes telles quelles — première écriture sur disque, aucune transformation. Format-agnostique (JSON, CSV...).
Stage	Staging/	Aplatit et normalise (tables normalisées, colonnes harmonisées comme ext_id, délimiteurs uniformisés, dédoublonnage des ré-exports).
Transform	Data Warehouse/	Modélise en schéma en étoile (faits + dimensions) et écrit au format Parquet — couche prête pour l'analyse.

Data Lake et **Data Warehouse** ne sont donc pas un seul et même espace : le Data Lake est un dépôt brut, fidèle à la source, non typé et non modélisé ; le Data Warehouse est la couche finale, typée, dédoublée et structurée en schéma en étoile — c'est la seule couche que consomment directement les dashboards et le modèle de prédiction.

Les dossiers Data Lake/, Staging/ et Data Warehouse/ sont régénérés à chaque exécution du pipeline et volontairement absents du dépôt Git (voir .gitignore) — gold/ et dashboard/data/ en sont des exports figés, commités, qui permettent d'explorer les résultats sans réexécuter le pipeline.

3. Pipeline ELT : Extract → Load → Stage → Transform

Le choix ELT (plutôt qu'ETL) est délibéré : les données brutes sont chargées telles quelles avant toute transformation, ce qui garde une trace fidèle de la source et permet de rejouer la normalisation sans jamais retoucher aux fichiers d'origine.



Figure 2 — Les quatre scripts de elt/, exécutés en séquence par spark-submit sur le cluster, chacun lisant la sortie du précédent.

Chaque phase est un script PySpark autonome, expédié aux exécuteurs du cluster via --py-files avec ses dépendances (config/logging_config.py, config/sources_config.py). Le détail de la modélisation appliquée lors de la phase Transform (schéma en étoile) fait l'objet d'un document dédié : data_modeling/Modelisation_star_schema.pdf.

4. Scalabilité horizontale : cluster Spark sur Docker Compose

Les phases Load, Stage et Transform ne s'exécutent pas en Python local : elles tournent sur un vrai cluster Apache Spark, démarré via Docker Compose (vm `cluster/docker-compose.yml`, image `apache/spark:3.5.9-python3`), avec un nœud *master* et plusieurs nœuds *worker* exécutés en parallèle (`docker compose up -d --scale spark-worker=2` (ou plus)).

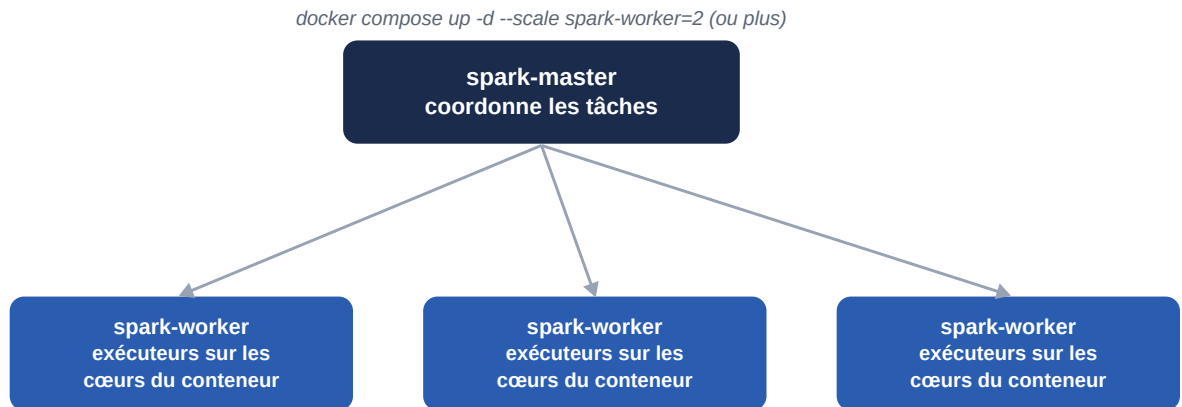


Figure 3 — Le master distribue chaque étape `spark-submit` (Load, Stage, Transform) sur *N* workers ; ajouter des workers augmente le parallélisme sans changer une ligne de code applicatif — c'est la définition d'une scalabilité horizontale.

Honnêteté sur le dimensionnement — le volume de données de ce projet (quelques centaines de milliers de lignes au total) ne justifie pas, à lui seul, l'usage de Spark : un simple traitement pandas en local aurait suffi et aurait été plus rapide à mettre en place. Le choix d'un cluster Spark conteneurisé est ici délibéré : il démontre une architecture capable de **monter en charge horizontalement** (ajouter des workers, pas changer le code) si le volume de données venait à croître significativement — un point que je souhaitais illustrer concrètement plutôt que simplement mentionner.

5. Orchestration et planification quotidienne

`orchestration/run_pipeline.py` enchaîne automatiquement Load → Stage → Transform sur le cluster, avec une **gestion de dépendance** : une étape ne démarre que si la précédente a réussi. L'exécution est planifiée chaque jour à 8h00 via une tâche Windows Task Scheduler enregistrée par `orchestration/register_scheduled_task.ps1`.



Figure 4 — Chaque étape est conditionnée à la réussite de la précédente ; en cas d'échec, la chaîne s'arrête et l'échec est journalisé plutôt que de continuer sur des données incomplètes.

Le script sépare volontairement `run()` (la logique d'enchaînement) de `main()`, qui enveloppe `run()` dans un `try/except` pour garantir que l'historique et les logs sont écrits même en cas de plantage inattendu — aucune exécution, réussie ou non, ne disparaît silencieusement.

6. Journalisation : logs et historique des exécutions

Deux mécanismes de traçabilité complémentaires accompagnent chaque exécution du pipeline :

- **logs/run_history.jsonl** — un historique append-only, une ligne JSON par exécution (date, durée, étapes réussies/échouées), qui permet de retracer la fiabilité du pipeline dans le temps sans avoir à parser les logs bruts.
- **log storage/YYYY-MM-DD.log** — une archive quotidienne du détail complet des logs de la journée, produite par `archive_daily_log()`, utile pour diagnostiquer une exécution précise en cas d'anomalie.

Sur le cluster, un piège classique de PySpark a été identifié et corrigé : un objet `Logger` ne survit pas au transfert (`cloudpickle`) vers les exécuteurs Spark. La solution retenue est d'appeler `get_logger()` à chaque fonction susceptible de s'exécuter sur un exécuteur, plutôt que de réutiliser un logger mis en cache au niveau du module.

7. Modélisation en schéma en étoile

La couche Data Warehouse organise les données en **schéma en étoile** : des tables de faits portant les mesures (montants) au centre, entourées de dimensions partagées portant le contexte descriptif (quel match, quel client, quel jour).

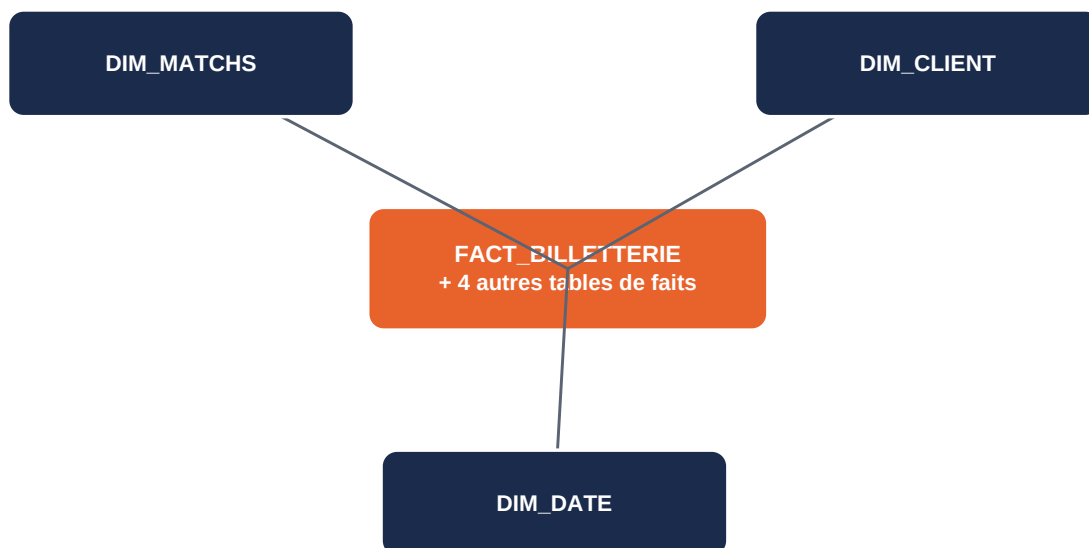


Figure 5 — Cinq tables de faits (`fact_billetterie`, `fact_buvette`, `fact_boutique`, `fact_commandes`, `fact_paiements`), chacune à son grain naturel pour éviter tout «fan-out», partagent les mêmes dimensions `dim_matches`, `dim_client`, `dim_date`.

Ce choix a permis, entre autres, d'identifier et corriger un bug de démultiplication («fan-out») causé par des commandes ré-exportées en double, et de trancher empiriquement le sens contre-intuitif du champ `is_cancelled` sur les paiements (`True` = tentative échouée, pas une vraie annulation). Le détail complet — grain de chaque table, décisions de modélisation, avantages du schéma en étoile — est documenté séparément dans `data modeling/Modelisation_star_schema.pdf`, pour ne pas dupliquer ce contenu ici.

8. Modèle de prédiction (Machine Learning)

`ml/train_model.py` entraîne un modèle de régression **Ridge** (scikit-learn) pour estimer le revenu d'un match à partir de sa compétition, son lieu, son mois et si c'est un jour de week-end — quatre variables encodées via un `ColumnTransformer` (`OneHotEncoder` pour les variables catégorielles).

Indicateur	Valeur	Signification
Matches d'entraînement	41	L'intégralité de la saison disponible.
R^2	0,78	78 % de la variance du revenu expliquée par ces 4 variables.
MAE (validation LOO)	≈ 109 301 €	Erreur moyenne absolue, soit ~21 % du revenu moyen par match.

Avec seulement 41 matches, un split train/test classique aurait gaspillé des données déjà rares pour l'évaluation. Le modèle est donc validé par **leave-one-out cross-validation** (`cross_val_predict` avec `LeaveOneOut()`), et l'hyperparamètre de régularisation (`alpha`) est lui-même sélectionné par validation croisée interne (`RidgeCV`) — une évaluation volontairement prudente plutôt qu'optimiste.

Ce modèle alimente à la fois l'onglet *Prédiction de revenu* du dashboard Streamlit et — coefficients recalculés à l'identique en JavaScript — le simulateur intégré au dashboard HTML autonome (voir sections 9 et 11).

9. Application Streamlit : tableaux de bord interactifs

`dashboard/app.py` est une application Streamlit unique, organisée en trois onglets, déployée sur Streamlit Community Cloud. Les données affichées proviennent du Data Warehouse en local, ou de l'instantané `dashboard/data/` lorsqu'il est exécuté à partir du dépôt Git seul (sans pipeline relancé).

Onglet « Revenu par match »

KPIs (revenu total, nombre de matchs, revenu moyen, meilleur match), filtres par compétition et période, graphique empilé billetterie/buvette, et tableau détaillé des 41 matchs avec taux de remplissage.

Revenu par match

Billetterie + buvette, saison 2025-2026 — données en direct (Data Warehouse local). Le revenu boutique n'est pas attribuable à un match précis (voir la note en bas de page) et n'est pas inclus ici.

Compétition

CHAMPIONNAT NATIONAL

COUPE EUROPEENNE

PLAYOFFS CHAMPIONNAT

Période

2025/10/09 – 2026/06/23

Revenu total

21 094 901 €

Matches

41

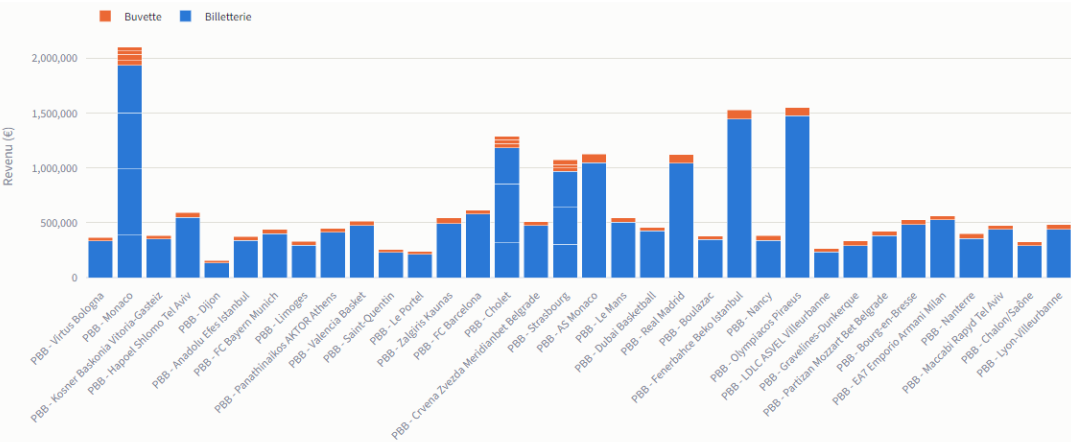
Revenu moyen / match

514 510 €

Meilleur match

PBB - Olympiac...

↑ 1 549 068 €



Détail par match

Date	Match	Compétition	Lieu	Billetterie (€)	Buvette (€)	Total (€)	Billets vendus	Taux de remplissage
09/10/2025	PBB - Virtus Bologna	COUPE EUROPEENNE	ADIDAS ARENA	334982 €	30901 €	365883 €	5310	66%
12/10/2025	PBB - Monaco	CHAMPIONNAT NATIONAL	ADIDAS ARENA	388227 €	43104 €	431331 €	6878	86%
15/10/2025	PBB - Kosner Baskonia Vitoria-Gasteiz	COUPE EUROPEENNE	ADIDAS ARENA	352816 €	31166 €	383981 €	5661	71%
17/10/2025	PBB - Hapoel Shlomo Tel Aviv	COUPE EUROPEENNE	ADIDAS ARENA	550557 €	40831 €	591387 €	7772	97%
19/10/2025	PBB - Dijon	CHAMPIONNAT NATIONAL	ADIDAS ARENA	133928 €	20200 €	154128 €	3317	41%
28/10/2025	PBB - Anadolu Efes Istanbul	COUPE EUROPEENNE	ADIDAS ARENA	340566 €	33267 €	373832 €	6107	76%
06/11/2025	PBB - FC Bayern Munich	COUPE EUROPEENNE	ADIDAS ARENA	400241 €	37672 €	437913 €	6500	81%

Capture d'écran — onglet Revenu par match.

Onglet « Prédiction de revenu »

Un simulateur en 3 étapes : choisir un adversaire pour Paris Basketball, ajuster le scénario (compétition, lieu, mois, jour), puis prédire — avec la fiabilité du modèle rappelée en tête d'onglet.

Prédiction de revenu

Choisis un adversaire déjà rencontré cette saison, ajuste les paramètres du match (ils ne sont pas obligés de correspondre à la saison prochaine — c'est un simulateur), et prédis le revenu billetterie + buvette attendu.

> Fiabilité du modèle

1. Choisir un adversaire pour Paris Basketball

Choisis un adversaire pour Paris Basketball

AS Monaco

Match du 15/01/2026 — revenu réel : 1 123 764 €

2. Ajuster le scénario

Compétition

COUPE EUROPEENNE

Lieu

ACCOR ARENA

Mois

Janvier

Jour

Semaine

3. Prédire

Prédire le revenu

Capture d'écran — onglet Prédiction de revenu.

10. Copilote IA : intégration de l'API Mistral

Le troisième onglet, « Copilote IA », propose deux boutons — « Revenu par match » et « Prédiction de revenu » — qui déclenchent chacun un résumé en langage naturel généré par l'API Mistral (SDK `mistralai v2.9.3`), à partir du contexte déjà calculé dans les deux autres onglets (partagé via `st.session_state`, puisque Streamlit ré-exécute tous les onglets à chaque interaction).

Résumé

Revenu par match

Prédiction de revenu

Capture d'écran — onglet Copilote IA.



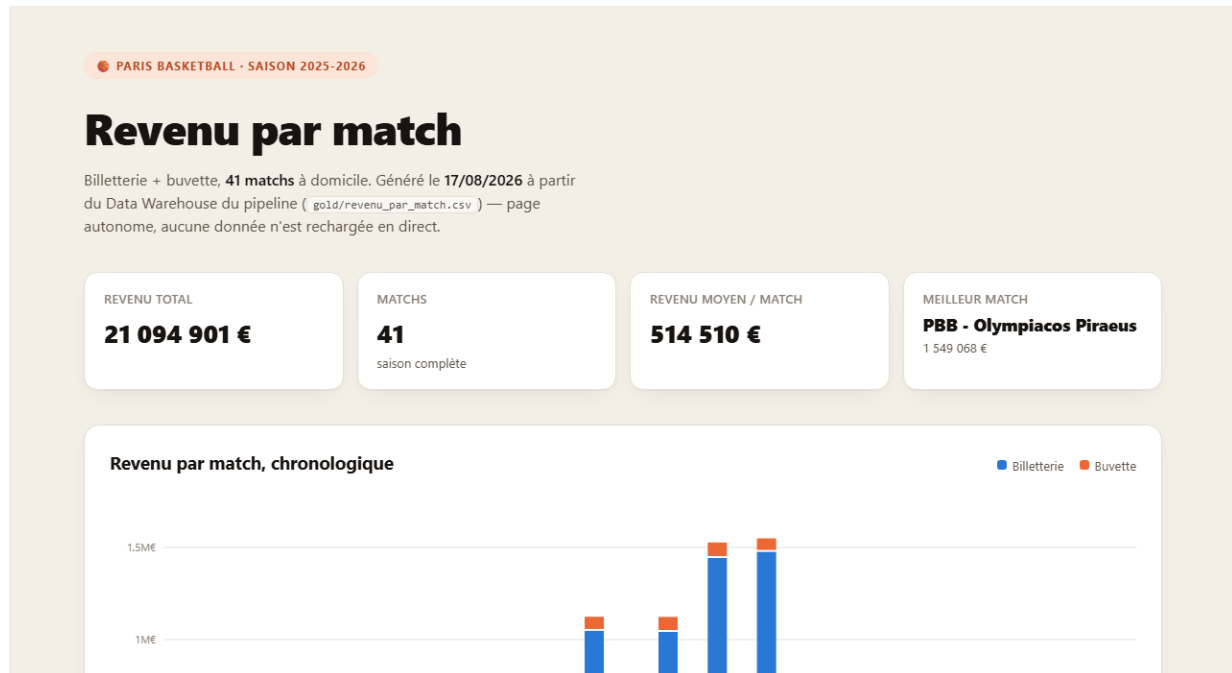
Figure 6 — Le contexte (chiffres déjà calculés côté application) est envoyé dans le prompt ; l'API ne recalcule rien, elle reformule en langage naturel.

Gestion de la clé API — la clé Mistral est fournie via `dashboard/.streamlit/secrets.toml`, un fichier gitignored (seul `secrets.toml.example`, sans valeur réelle, est commité). L'appli reste fonctionnelle sans clé configurée : `get_mistral_client()` encapsule la lecture de `st.secrets` dans

un try/except (qui lève une exception dédiée si aucun fichier de secrets n'existe du tout) et affiche un message explicite à la place du résumé plutôt que de faire planter l'application.

11. Dashboard HTML autonome

dashboard.html, à la racine du projet, est un livrable pensé pour être consulté en quelques minutes sans rien installer : aucun serveur, aucune dépendance, un simple double-clic l'ouvre dans un navigateur. Les données (41 matchs) sont figées dans le fichier au moment de sa génération, à partir de gold/revenu_par_match.csv.



Capture d'écran — en-tête, KPIs et graphique.

Le fichier réimplémente aussi la prédiction de revenu, en JavaScript pur : les coefficients exacts du modèle Ridge entraîné (section 8) sont copiés dans le fichier, permettant de simuler un scénario sans serveur ni appel réseau — vérifié pour reproduire à l'euro près les prédictions du modèle Python d'origine.

Prédiction de revenu

0,78 ± 109 301 € 41

R² (validation LOO) erreur moyenne (~21 %) matchs d'entraînement

Régession Ridge sur compétition, lieu, mois et jour de semaine — volontairement simple vu le faible nombre de matchs. À utiliser pour comparer des scénarios, pas comme un chiffre garanti (détail dans NOTES.md).

Choisis un adversaire pour Paris Basketball, ajuste le scénario si besoin, puis prédis.

ADVERSAIRE	COMPÉTITION	LIEU	MOIS	JOUR
Anadolu Efes Istanbul	Coupe Européenne	Adidas Arena	Octobre	Semaine

Prédire le revenu

Capture d'écran — section Prédiction de revenu.

Le fichier gère nativement les thèmes clair et sombre (variables CSS, respect de prefers-color-scheme), vérifié par capture d'écran dans les deux modes avant livraison.

12. Autres éléments de l'architecture

Au-delà des points ci-dessus, plusieurs choix d'architecture et de méthode soutiennent l'ensemble du projet :

- **gold/** — export CSV de toutes les tables du Data Warehouse, dont `revenu_par_match.csv` pré-agrégé, pour vérifier un chiffre dans un tableur sans rien exécuter.
- **Environnement reproductible** — un environnement virtuel Python dédié (`.venv/`) avec versions figées dans `requirements.txt` ; résolu un conflit de dépendance (`pyarrow/streamlit`) découvert seulement au déploiement sur Streamlit Cloud, corrigé en alignant la version locale sur le résolveur plus strict du cloud.
- **Gestion de version** — historique Git complet, dépôt public sur GitHub (`Elijoelemile/Analyse_et_Prediction_de_Revenus_de_Paris_basketball...`), utilisé aussi pour le déploiement continu de l'application Streamlit.
- **Tests automatisés** — la logique du dashboard Streamlit est testée sans navigateur via `streamlit.testing.v1.AppTest`, notamment pour vérifier le comportement du copilote IA en l'absence de clé API.
- **Sécurité et secrets** — aucune clé ni donnée brute committée : `.gitignore` exclut `Data Lake/`, `Staging/`, `Data Warehouse/`, `logs/` et le fichier réel de secrets ; seuls des exports figés et des exemples de configuration sont versionnés.
- **Livrables de restitution** — `README.md` (installation et exécution), `NOTES.md` (question, chiffres clés, réponse, limites assumées) et le présent document, pensés pour être lus indépendamment du code.

13. Synthèse : les outils et leur rôle

Outil / Technologie	Rôle dans le projet
Python (pandas, PySpark)	Extraction, normalisation et transformation des données.
Apache Spark	Traitement distribué des phases Load / Stage / Transform.
Docker Compose	Cluster Spark conteneurisé (1 master + N workers) — scalabilité horizontale.
Parquet	Format de stockage typé de la couche Data Warehouse.
Windows Task Scheduler	Déclenchement quotidien automatique du pipeline (8h00).
scikit-learn	Modèle de prédiction de revenu (régression Ridge, validation LOO).
Streamlit	Application web interactive (3 onglets), déployée sur Streamlit Community Cloud.
API Mistral	Résumés en langage naturel dans le Copilote IA.
HTML / CSS / JavaScript	Dashboard autonome sans serveur, avec prédiction réimplémentée.
ReportLab	Génération des documents PDF de documentation (celui-ci et le schéma en étoile).
Git / GitHub	Gestion de version et hébergement du dépôt.

Chacun de ces outils a été choisi pour un rôle précis plutôt que par défaut — en particulier Spark/Docker, dont l'intérêt ici est démonstratif (section 4) plus que strictement nécessaire au volume de données traité.